Auditing the ARC-AGI-3 Public Benchmark: Scoring Rule, Environments, and Human Baselines

Ibrahim Mian Shayaan Siddique
Millennium Research
{ibby,shayaan}@millenniumresearch.ai

Abstract

ARC-AGI-3 evaluates agents on interactive environments by comparing their action counts against per-level human baselines. We audit the public half of that apparatus: the 25 public demonstration environments, the open toolkit and benchmarking harness, and the published scoring rule. Our method is reject-only. We write a machine-verified model of the rule under audit, compile it, and run it differentially against the shipped implementation; every component either reports a disagreement or reports no disagreement, and a human reads each disagreement before it is called anything.

We find no defective environment, and the level-scoring formula computes what its prose defines. The finding that bears on a scored run is in the benchmarking harness. After a game over the harness issues a `RESET` of its own, and that reset consumes the agent's per-level action budget. We show by a paired counterfactual, identical in policy and differing only in whether the forced reset is charged, that this can end a level the agent would otherwise have completed: at one budget the shipped harness scores the environment `0.0` where the counterfactual completes the level. The effect is not a knife edge. We predicted before measuring, and confirm, that the set of budgets on which the two disagree is exactly as wide as the number of game overs, so an agent's effective per-level budget is $\lceil 5b \rceil$ minus the number of times it died. No published document states that these resets happen or that they are charged. We also find one arithmetic defect above the scoring formula and four documentation problems. On the toolkit's local scoring path — the one its own quickstart uses — environment scores are averaged over the environments a scorecard contains, where the report defines the total as divided by the total number of environments: a scorecard covering three environments of a 135-environment set reports 100% where the documented rule gives 2.22%, and an environment that fails leaves the divisor rather than scoring zero. The official harness runs the toolkit in a mode where the server supplies the scorecard, so that arithmetic does not run for an official run. The technical report's Equation (1) caps the efficiency ratio before squaring, permitting a per-level score of 132.25%, where its own prose, the documentation and the code all cap the score at 115%. The five-times action budget the report describes exists only in the benchmarking harness; the scorer applies no cutoff, so a scorecard produced by any other agent is scored without it. The published `/api/games` schema omits a field the endpoint returns and the toolkit reads. And the 342 human replays announced as open source in April 2026 could not be located through the Foundation's own public channels. We also report one unreached robustness edge in the scorer, and a definitional point: for a level whose graph we enumerate exhaustively, the report's stated win probability of "exactly 1 in 355" matches our computation to three significant figures under a random-policy reading it does not state.

Positively, we verify two levels of the public environment `ls20` in Dafny and find the models and the shipped implementation agree on every one of 190,560 transitions; we probe reset behaviour in all 25 environments and find that a level reset restores the frame an agent observes on all 18,663 probes; and we check the published human baselines against exact optima, finding all six resolvable levels consistent. Everything is reproducible from a public repository at pinned commits.

1 Introduction

ARC-AGI-3 [1] is an interactive-reasoning benchmark: an agent is placed in a turn-based environment whose rules are stated nowhere, and must discover them. Scores are efficiency ratios. For each level of each environment, the agent's action count is compared against a human baseline, the ratio is squared and capped, the level scores are weighted by level index, and the environment scores are averaged. The benchmark is unusual among evaluation suites in how much of it is public: the toolkit, the harness, the example agents, 25 of the 135 environments, and a technical report describing the scoring rule in detail.

That openness makes an audit possible, and an audit worthwhile. A score under this rule depends on three artefacts being what they claim: the environments, the human baselines they are measured against, and the scoring code. This paper checks all three, as far as each admits checking.

Our position is not adversarial. Benchmarks are infrastructure, and the useful service an outside reader can perform is to say precisely which parts have been checked, by what method, and what survived. Where we found nothing wrong we say so at the same length as where we found something.

Contributions.

1. A machine-verified model of the published scoring rule, compiled and run differentially against the shipped scorer on preregistered planted traces (Section 4). The scorer implements the rule as the report's prose states it; four documentation problems and one unreached code edge are reported.

2. Machine-verified models of two levels of a public environment, agreeing with the shipped implementation on every transition of their complete reachable state graphs (Section 5). We also give the first published exhaustive state counts for those levels and check the report's own stated win probability against them.

3. A measurement of which environments admit state-graph methods at all (six of 25), and reset and level-accounting probes covering all 25 (Sections 6 and 7).

4. A check on the published human baselines that requires no access to the human data: a baseline cannot be below the level's optimum, and breadth-first search bounds that optimum from below (Section 8).

2 Scope, and what we did not do

In scope: the 25 public demonstration environments, the toolkit `arcprize/ARC-AGI` at commit `f12822c`, the harness `arcprize/arc-agi-3-benchmarking` at `1aa78da`, the example

agents `arcprize/ARC-AGI-3-Agents` at `4743e7d` (all MIT-licensed), the technical report, and the public documentation.

Out of scope, deliberately and throughout: the 55 semi-private and 55 private environments. We made no attempt to recover, probe, scrape or infer them. All environment execution was local and offline. No component of this work is an agent whose purpose is to score on the benchmark; the random player of Section 7 is an instrument for probing reset semantics and never attempts to win, and did not win anything.

We audit the *shipped local* artefacts at the pinned commits. The server-side scorer that produces the official leaderboard is not observable to us, and we make no claim about it.

3 Method

Four commitments shape everything below.

Reject-only. No automated component of this work emits a verdict of correctness. Each either reports a disagreement or reports that it found none, and a human reads every disagreement and classifies it before it is called anything. "No disagreement on N fixtures" is a statement about N fixtures.

Preregistration. The bar, the predictions, and the reading of each possible outcome were written down and committed before each measurement. Deviations are dated in a decisions file and disclosed. One preregistered prediction was wrong and is reported as such in Section 4.

Machine-verified models, run differentially. For each rule under audit we write a model in Dafny [5] whose properties are proved rather than tested, compile it to JavaScript, and compare it against the shipped implementation on traces recorded from that implementation. A model carries no `assume`, `{:axiom}`, `{:verify false}` or `{:extern}`, and a proof obligation that times out is treated as unknown, never as discharged. The models of Section 5 are generated from the shipped level data by a script, so a hand edit fails the test suite.

Claims discipline. Every number in this paper is produced by a script into an artefact and registered in a claims file, and a checker verifies that each value appears both in the text and on a matching line of the log that produced it. The repository ships the scripts, the artefacts and the checker.

4 The scoring rule

4.1 What the rule says

Report v2 §4.1 defines the level efficiency score for level l of environment e, with $a_{l,e}$ the agent's action count and $h_{l,e}$ the human baseline, and gives the environment score as a level-index-weighted average subject to a cap at the weighted fraction of levels completed. The human baseline is the upper-median action count of first-time human players; report v1 said second-best, and the change is documented in an April 2026 post [2].

4.2 An inconsistency between the equation and the prose

The report typesets the level score as

$$S_{l,e} \;=\; \min\!\left(1.15,\; \frac{h_{l,e}}{a_{l,e}}\right)^{2},$$

which caps the *ratio* before squaring and therefore admits a per-level score of $1.15^2 = 1.3225$. The same section's prose says the per-level efficiency is capped "at 1.15x human baseline" and that a level score lies between 0% and 115%; the documentation says the same; the toolkit's changelog records the change from a 100% cap to a 115% cap; and the shipped code applies `min(score, 115.0)` after squaring. Every artefact except the equation caps the score.

The difference is observable. Our planted trace `P2c` has one level solved in 5 actions against a baseline of 10 and three solved in 12: the shipped scorer returns an environment score of `49.333333`, the prose reading returns the same, and the equation reading returns `50.483333`. A level solved in 9 actions against a baseline of 10 scores `115.0` in the shipped code and 123.46 under the equation.

4.3 The action budget is not part of the scoring rule

Report v2 §4.3.1 states an action budget of five times the human baseline per level, after which the agent is terminated. We find that budget implemented only in the benchmarking harness, where every shipped model configuration sets a multiplier of 5.0 and the per-level budget is enforced in the agent's termination test. The scorer applies no cutoff at all. Our planted trace `P3b`, a level completed in 51 actions against a baseline of 10, is scored `93.589645` by the shipped scorer, where treating the budget as part of the rule gives `0.000000`.

Neither the documentation's methodology page nor its competition-mode page mentions the budget. The consequence is narrow but real: a scorecard computed by the toolkit from a run that did not use the official harness — an example agent, a human session, a third-party agent — is scored without the cutoff the report describes.

We verified separately that the harness enforces the budget exactly. Across six scripted runs against the toolkit in offline mode, driving the harness's own control loop with a scripted model, no level ever exceeded its budget, the harness's action counter and the scorecard's agreed on every run, and two runs exited on the budget precisely at its boundary.

4.4 The aggregation denominator

The scoring rule of §4.1 governs one environment. Above it the toolkit computes an overall score, and there the denominator is the number of environments the scorecard contains — the number actually played — rather than the number in the set. The report defines the total as "the sum of individual environment scores divided by the total number of environments", and the documentation's methodology page describes it as the average of all game scores.

On a planted scorecard covering three environments of a set of 135, each completed at exactly the human baseline on every level, the toolkit reports a total of 100.0 where the documented rule gives 2.222222222, a factor of 45.0. Played over a whole set the two agree exactly, so the difference is precisely the partial-coverage case.

Which runs this reaches is measured rather than assumed. Calling `Arcade.get_scorecard` under each operation mode, with a transport that refuses to open a socket, identifies where the number comes from: `offline` and `normal` compute it locally through this arithmetic, while `online` and `competition` take a *remote fetch (server supplies the scorecard).* The official benchmarking harness constructs the toolkit as `Arcade(operation_mode=OperationMode.ONLINE)`, so this arithmetic does not run for an official run at all. The server-side scorer is not observable to us and we make no claim about it, about the official leaderboard, or about any published result.

The exposure is the local path, and it is not hypothetical. The toolkit's own minimal example constructs `Arcade()` — the default mode is `normal` — plays a single environment and prints `scorecard.score`. Following the quickstart on one environment prints that environment's score where the report's definition gives it divided by the size of the set.

A second aspect sharpens it. Our first reading supposed that covering the whole set is safe; on the local path it is not, because the divisor counts environments that produced a card. Three environments played perfectly plus a fourth that produced no card — a failure, a skip, an id that did not match — gives a total of 100.0 where the documented rule over four gives 75.0. An environment that produces a card and completes nothing, and one opened but never played, both remain in the divisor and give 75.0 correctly. The boundary is between a card that exists and one that does not.

Relatedly, environment information is matched by full game identifier including version. A scorecard recorded against one version and scored against a listing carrying another scores that environment 0.0, with the message *No Matching EnvironmentInfo found.* That moves a score down rather than up, and a version refreshing between a run and its scoring would silently zero it.

4.5 Resets are charged to the agent, and the human side is unknown

A level reset increments both the reset count and the action count in the scorecard, and the action falls on the level in progress, so it lowers that level's efficiency. On an otherwise perfect five-level play, one reset before each of levels two to five takes the environment score from 100 to 83.801652893.

No document we found states whether a reset counts as an action. The methodology page defines an action as a discrete interaction that affects the game state and excludes internal operations, which does not settle it. The question has teeth because human participants were explicitly permitted to reset a level at any time, and the report observes that some "reset levels after reaching a solution in order to improve efficiency". If the published baselines were gathered without charging human resets while agents are charged for theirs, every ratio is biased against the agent by an amount depending on how often each side reset. We cannot settle it. Doing so needs the human replays, which we were unable to locate (§8.3). It is the sharpest open question we found in the scoring rule.

4.6 Resets the agent did not choose

§4.5 concerns resets an agent chooses. The benchmarking harness also issues them on its own: after a game over it returns `RESET` rather than consulting the model, because a reset is

the only way to continue. Those resets are counted by the harness like any other action and, on the local path, charged by the scorer to the level in progress.

Driving the harness's own control loop against the toolkit's fixture game, an agent that loses a level three times and then solves it chooses 16 actions and is counted 19, and the level is scored on all 19. Against that level's baseline of four, being scored on 19 rather than on the 16 the agent chose lowers the level's contribution by about a fifth.

A fall in a completed level's score is the mild consequence. The counter these resets increment is also the one the harness stops on. The per-level budget is $\lceil 5b \rceil$ for a baseline b; `is_done` ends the level once the counter reaches it; `choose_action` returns a forced action before the ordinary increment, but the routine that records the forced action increments the same counter regardless. A run can therefore be cut off on an action the agent did not choose.

We measured that as a paired counterfactual rather than arguing it from the code. The same scripted policy runs twice against the same game with the same budget, differing in one respect: in the second run the forced reset still happens and still reaches the environment, but does not increment the budget counter. At a budget of 8 the shipped harness exits on the budget with the level incomplete and the environment scoring `0.0`, while the counterfactual completes the level and scores `1.316872428`. The agent's play is identical in both. The reset it did not choose is the entire difference between a completed level and a failed one.

This is not a knife edge, and we predicted its shape before measuring it. The counterfactual completes the level as soon as the budget covers the agent's chosen actions, while the shipped harness additionally needs one action per game over, so the set of budgets on which the two disagree should be exactly as wide as the number of game overs. It is, in every case probed: no denial budgets for an agent that never dies, `[8]` for one death, `[12, 13]` for two, `[16, 17, 18]` for three. The general statement is simpler than the demonstration. *An agent's effective per-level budget is* $\lceil 5b \rceil$ *minus the number of times it died.* Each death quietly costs an action of allowance on top of the progress it loses, and the exposure widens with every one.

Applied to the real public set, one such reset costs a level between 0.03 and 3.33 per cent of its budget, and lowers an otherwise perfect completion by a median of `3.251814028` points across the 183 levels of the 25 public environments. The worst case is the smallest baseline in the set, `sc25` level 1 at $b = 6$, where one forced reset takes a perfect completion from `100.0` to `73.469387755`.

Two things this is not. It is not a claim about the server, which is not observable to us and may charge such a reset differently, or enforce the budget differently, or not at all. And it is not a design criticism: the harness has to send something after a game over. What is absent is any statement to an entrant that these resets happen, that they consume the per-level action budget, and that they can end a level the agent would otherwise have completed. It compounds §4.5: the human baselines' treatment of resets is unknown, and here the agent is additionally charged for resets it never chose.

Several negatives from the same instruments are worth recording, because each rules out a way a client can distort its own score. A retried model call never reaches the environment: the harness's retry path contains no call that does, and across every scripted run the number of environment calls equals the number of actions counted. One intended action is sent exactly once: driven with a transport that records every attempt and opens no socket, the remote wrapper makes a single attempt under a connection error, a read timeout after the request

was sent, a server error and an empty response, so it never has the server count two actions for one intent. A level that advances on the last action the budget permits is not cut off. The frame cap subsamples evenly rather than truncating and always keeps the last frame, so the settled frame is never dropped; whether the humans who set the baselines saw more is not establishable from the published material and we do not compare. And the reset the wrapper issues when the environment is constructed goes on the wire but is counted by neither the harness nor the local scorer, which treats it as beginning a play; the server's treatment of it is unknown.

One further limit deserves mention without being a finding. Besides the action budget, a run also stops on a wall-clock limit that the report does not describe. It is uniform: none of the twelve shipped model configurations overrides it or the frame cap, and all set the same action multiplier, so the base of twelve hours for a single environment applies to every entrant. We entered the branch by setting the limit to zero rather than by waiting, and confirmed it ends a run, but at twelve hours per environment we could not show it binds.

4.7 An unreached edge in the scorer

The engine permits a game to advance the level counter twice within a single action. Where that happens, the scorer pairs the k-th recorded level transition with level k and scores the final level as not completed even though the play ends in a win: our trace `P8` returns `66.666667` where the documented rule gives `100.000000`.

We then asked whether any public environment can trigger it. Statically, all 25 sources contain exactly one `next_level()` call site and none is inside a loop. Dynamically, we found no such action anywhere: not in either complete state graph of Section 5, not in the 1,245,427 transitions enumerated in Section 6, and not in the 479,040 actions played in Section 7. We report it as a robustness edge that a guard would close, not as a defect with consequences.

4.8 What the scorer gets right

On 14 preregistered planted traces — 13 of which have an oracle value, the fourteenth exercising a case the rule leaves undefined — the shipped scorer agrees with our verified model of the prose rule on 12, both on environment scores and on every per-level score, disagreeing on one: the double-advance edge above. The agreements include the 115 cap, one-indexed level weights, the environment cap at the weighted completed fraction, best-of-plays selection, and a level solved in exactly the baseline. The model itself discharges 50 proof obligations with no errors and no timeouts.

Two smaller documentation items: the code comments beside the cap still say "max 100", predating the change to 115; and the published response schema for `/api/games` omits `baseline_actions`, a field the endpoint does return and the toolkit reads from it.

A wrong prediction, disclosed. We preregistered that the harness would issue and count an initial reset, making it one action stricter than the scorer on the first level. It does not: both toolkit wrappers issue that reset inside environment construction, and neither the harness nor the scorecard counts it. We added a further probe to exercise the boundary the original one had missed, and report the corrected picture: the two counts agree.

	`ls20` level 1	`ls20` level 2
States in the complete reachable graph	14,337	34,739
Transitions	56,772	133,788
Minimum actions to complete	13	45
Published human baseline	22	123
Proof obligations discharged (0 errors)	17	26
Disagreements, model vs implementation, all transitions	0	0
Trace steps compared (30 traces)	3,208	1,903
Disagreements on traces	0	0
`RESET` returns to the level start	500/500	500/500
Actions advancing the level counter by two	0	0

Table 1: Two levels of the public environment `ls20`, audited against machine-verified models of their transition rules.

5 Environment models

5.1 Method

We selected one public environment by a rule fixed in advance: the fewest lines of transition logic among environments with at least six levels and a mechanic described in the report or documentation. Only `ls20` satisfies the last clause — the report shows its first level as a state graph and states a win probability for it — so it was selected even though eleven environments have smaller sources. All 25 environment sources are identifier-obfuscated, so the models are written from behaviour and structure rather than from names.

For each level we extract the constants from the shipped level object by script (walls, start, goal, modifier tiles, energy pickups, step budget, decrement, lives), hand-write the transition rules from the shipped step function at the rule level, and generate a Dafny model. We then enumerate the level's reachable state graph directly from the shipped game, and compare the compiled model against the implementation on every transition of that graph and on 30 recorded traces.

5.2 Results

Table 1 summarises. Both models verify with no errors and no timeouts, and agree with the shipped implementation everywhere.

The verified properties are: the level is winnable from its own start state, witnessed by the shortest action sequence the enumeration found and replayed through the model with every intermediate state asserted equal to the state the shipped game is actually in; no action leaves the legal state space; and reset restores the start. The first is a stronger statement than "the run ends in a win": if model and implementation parted company anywhere along the winning path, it would not verify.

5.3 The report's stated win probability

Figure 3 of the report shows the first level of `ls20` as a graph, notes "three repeating states – an artifact of the three-life mechanic", and states that the win probability for that level is "exactly 1 in 355". The term is not defined. Our exhaustive enumeration gives a graph that is indeed three copies of one structure, one per remaining life, and under the reading that seems intended — the probability that a uniformly random policy over the four movement actions reaches the win before a game over, starting from the level's reset state — we compute `0.002813847150161035`, that is 1 in `355.38533070027296`. That agrees to three significant figures; "exactly" is a rounding. Two other readings we computed, one over the number of states and the winning fraction of states, both give 1 in 14,337. We report this as a definitional gap rather than an error: the environment agrees with the claim under its natural reading.

5.4 What counts as a state

A state count depends on the identity function used, and the choice is not neutral. At the rule level the three-life structure is symmetric on both levels: distinct states by remaining life are 4732/4731/4731 on level 1 and 7400/7399/7399 on level 2. Our enumerator's raw key, which is deliberately finer than the rendered frame, reports 7400/13024/13024 on level 2. The cause is that a life loss re-appends previously consumed pickups to the end of the level's sprite list, so a run that consumed one and a run that did not are distinguishable by internal ordering while rendering identically; level 1 has no pickups and is symmetric under both measures. This changes no transition and no absorption probability, but it is a caveat on any published state count for these environments, including the report's figure and our own: a frame hash, an engine-state hash and a rule-level abstraction give three different answers.

6 Which environments admit state-graph methods

Before enumerating anything we measured what is enumerable. Nineteen of the 25 public environments advertise the click action, whose x and y each range over 0 to 63, giving at least 4,096 successors per state. Exhaustive enumeration is out of reach for those within any reasonable budget. The remaining six — `ls20`, `tr87`, `tu93`, `g50t`, `re86`, `wa30` — advertise four or five simple actions.

This is a measured property of the advertised action space, taken from the shipped games, and it bounds any state-graph method applied to ARC-AGI-3, ours and the published ones alike.

For those six we enumerated level 1 by depth-first search, which holds only the objects along one path and so is bounded by depth rather than by the width of a search layer. Two completed within budget, `ls20` at 14,337 states and `tu93` at 2,609; the other four stopped at a deterministic cap of 150,000 states, which establishes nothing about reachability either way and is reported as such. Across 616,946 states and 1,245,427 transitions: no level was shown unwinnable, `RESET` returned to the level's start on all 3,000 probes, and no action advanced the level counter by two.

7 Reset behaviour across all 25 environments

Two of the three questions the enumeration answers need no state graph, so they can be asked of the nineteen environments enumeration cannot reach: does `RESET` restore the state the level began in, and can one action advance the level counter by two. We answer both by playing each environment at random inside its own advertised action set, sampling click coordinates uniformly within the declared bounds, and probing as we go.

We measure the reset question twice, because two different relations are at stake. *Frame identity* asks whether the rendered grid an agent observes returns to the level's opening frame. *Engine-state identity* asks whether the game object does. Conflating them turns a property of the instrument into a claim about an environment.

Across 18,663 probes in all 25 environments, the rendered frame after a level reset was identical to the frame the level began with every single time. Engine state matched on 15,440 of the same probes; in seven environments (`cd82`, `ft09`, `m0r0`, `r11l`, `sb26`, `sc25`, `sp80`) a small number of the game object's own attributes survive a level reset. Several are read back by the games' own code, so they could in principle influence later behaviour, but none is visible in the frame at reset time.

We do not report this as a defect. ARC-AGI-3 states no rules for any environment by design, so an environment that leaves an internal counter alone contradicts nothing. It does bear on a documented assumption: the report's own graph construction begins "from the reset state of a selected level", and published graph-based agents identify states by hashing the frame. Frame identity and engine-state identity are not the same relation, and in seven of 25 public environments they disagree about whether a reset returns to a canonical state. Anyone building on "the reset state of a level" should say which they mean.

Over 479,040 actions in all 25 environments, no action advanced the level counter by two and none moved it backwards outside a full reset.

Why these negatives are readable. A detector that cannot fire proves nothing. Both are tested against deliberately broken games: with state injected that a reset does not restore, the probe reports a mismatch while the frame still matches; with a reset made to leave a visible change, the frame counter falls; and with the engine made to advance two levels in one action, the detector catches it.

8 The human baselines

8.1 A check that needs no human data

Every ARC-AGI-3 score is a ratio against `baseline_actions[l]`. A human cannot complete a level in fewer actions than the level's optimum, so a published baseline must be at least that optimum. Breadth-first search supplies the optimum, and supplies a sound bound on it even when it does not finish: having expanded every state at depth d without reaching a win, it has proved no solution exists in $d + 1$ actions or fewer.

Two things were settled before measuring, either of which would have made a contradiction spurious. The scorer pairs `baseline_actions[level_idx]` with that level's own action count, computed as a difference between cumulative counts, so the published values are per

Environment	Level	Minimum actions	Published baseline
ls20	1	13	22
ls20	2	45	123
ls20	3	39	73
tu93	1	18	19
tu93	2	10	16
tu93	3	19	34

Table 2: Exact optima against published human baselines, for the levels where the search completed. Every optimum is replayed through the shipped game and shown to complete its level.

level. And the published values are not monotone — ar25 is $[32, 50, 75, 37, 89, 159, 233, 73]$ — so they cannot be cumulative.

8.2 Result

Of 18 levels attempted across the six enumerable environments, six resolved and all six are consistent; none is impossible; twelve were stopped by a time or memory cap after completing depths between 8 and 16 against baselines between 26 and 183, and establish nothing either way.

Table 2 gives the resolved levels. The tightest margin is tu93 level 1, where the upper-median first-time human used one action more than the optimum on an environment whose rules are stated nowhere.

We then attempted to resolve the remaining levels by changing the search, and could not. Breadth-first holds a whole layer of game objects, so memory was the obvious suspect; we added depth-limited depth-first search with shallowest-depth memoisation, which gives the same completeness guarantee to a stated depth while holding only the objects along one path, and an iterative-deepening form that banks a bound after each completed depth. Memory fell by roughly a factor of seven on the level we compared. It resolved nothing: in the same time budget breadth-first reached a greater depth on both levels compared, because deepening re-explores, and on levels breadth-first can finish it is strictly better. The binding constraint is not memory but the size of the state space at the depths these baselines occupy, and no exhaustive search we can write reaches depth 26, let alone 183. Settling these levels would need the human replays, which we could not locate, or a solver for these environments, which this audit's own scope excludes.

These optima are proved against the shipped local environment at the pinned version. That is not necessarily the environment the human study measured, and consistency with a local optimum says nothing about how the number was gathered.

8.3 The replays could not be located

An April 2026 post [2] states that the Foundation open-sourced the Public Demo dataset, "which includes 342 human step-by-step replays" across the 25 public environments. We were unable to find them. Checked on 4 September 2026: the arcprize GitHub organisation

lists ten repositories, none a human-replay dataset; the `arcprize` HuggingFace author lists three datasets, of which the only human-testing one is for ARC-AGI-2; and the single link the announcement provides is a shortener that did not resolve for us, returning HTTP 429 and then 403.

This is a statement about what these checks found on that date, not a claim that the data does not exist; it may well be reachable by a route we did not try. We report it because a link that does not resolve is worth an owner's attention, and because until the replays are reachable the baselines that every score depends on cannot be checked against the plays that produced them.

9 Limitations

A reader should hold the following in mind.

1. Everything here concerns the public toolkit, harness and documents at the pinned commits. The server-side scorer behind the official leaderboard is not observable to us and is not claimed to match. This bears hardest on §4.4: we report a defect in a public artefact, not a claim about anyone's published result, and we do not compute what any particular reported score would have been.

2. "No disagreement" is over the fixtures and transitions we ran, chosen by us. It is not a proof of correctness.

3. Our Dafny models encode our reading of the report and the shipped source. A disagreement could always be the model being wrong, and we treated that as the default explanation throughout.

4. Two levels of one environment are audited with verified models. The breadth results cover level 1 of six environments, two exhaustively. The nineteen click-based environments have no reachability result at all.

5. The random play probe won nothing and mostly stayed on level 1, so its negatives cover the states it visited.

6. A capped enumeration is a sample of a larger graph. Four of the six enumerable environments are represented by the first 150,000 states a depth-first search happened to visit.

7. Several of our own errors were found during the work and are recorded in the repository's decisions file rather than quietly fixed: a state key that ignored non-`ls20` game state, two hashing gaps that merged or spuriously split states, an off-by-one in the lower bound, and a peak-memory guard that let one heavy environment mark later ones as capped. Each invalidated results that were re-run.

8. What a reader must still trust: Dafny and its JavaScript backend, Node, the vendored toolkit at the pinned commit, and our reading of the English in the report and documentation.

10 Related work

Rudakov, Shock and Cowley [3] build hash-identified state graphs over ARC-AGI-3 environments in order to *solve* them, reporting no state counts, no win probabilities and no verification. Rodionov [4] builds executable Python world models for all 25 public games, checked against recorded observations rather than machine-verified, and does not audit environments for defects. To our knowledge this is the first machine-verified model of an ARC-AGI-3 environment's rules, the first exhaustive state counts published for specific levels, and the first check of the published human baselines against proved optima.

The broader practice this paper belongs to — auditing a benchmark's reference artefacts against what they claim, rather than competing on it — follows work on formal-statement benchmarks where reference defects were found and fixed by their owners.

11 Reproducibility and disclosure

The audit repository, released with this paper at [**REPOSITORY URL**], contains the preregistrations, the decisions file recording every deviation and every error of ours, a dated ledger, the generated models, the scripts, the artefacts, a test suite, and a claims file with a checker that verifies each number in this paper against the log that produced it. Environment sources are not redistributed; they are fetched from the ARC API by a script. The three audited repositories were archived at Software Heritage at intake.

This paper is published without prior private notice to the ARC Prize Foundation. We record that plainly rather than implying a disclosure process that did not happen. Two considerations decided it. Everything examined here is already public, and every defect is in published material or in the client an entrant runs on their own machine, so nothing in this paper tells a reader something they could not have derived from the same public sources; there is no exploitable secret to withhold. And the findings that change a number are properties of the shipped client, which any entrant can verify against their own run today. We have no reason to think the Foundation would disagree with any of them, and the repository contains everything needed to check every one.

Acknowledgements

The ARC Prize Foundation publishes the toolkit, harness, agents, technical report and documentation openly and under permissive licences. That is what makes an audit like this possible from outside, and it is not the norm. We thank them for it, and note that the great majority of what we checked was exactly what it claimed to be.

References

[1] ARC Prize Foundation. *ARC-AGI-3: A New Challenge for Frontier Agentic Intelligence*. arXiv:2603.24621, v2, April 2026.

[2] ARC Prize Foundation. *Measuring Human Performance on ARC-AGI-3*. `https://arcprize.org/blog/arc-agi-3-human-dataset`, 14 April 2026.

[3] E. Rudakov, J. Shock, and B. U. Cowley. *Graph-Based Exploration for ARC-AGI-3 Interactive Reasoning Tasks*. arXiv:2512.24156, December 2025.

[4] S. Rodionov. *Executable World Models for ARC-AGI-3 in the Era of Coding Agents*. arXiv:2605.05138, v2, June 2026.

[5] K. R. M. Leino. *Dafny: An Automatic Program Verifier for Functional Correctness*. LPAR-16, 2010.